

Where Memory Belongs: LEDGER, an Object Ledger for Memory-Augmented VLAs

Tanguy Dieudonné^{1,2*}, Jack B. Jedlicki^{1*}, Heng Yang¹

¹Harvard University, ²ETH Zürich

*Equal contribution.

tdieudonne@ethz.ch jackbjed@g.harvard.edu

Memory is essential for long-horizon, partially observed robotic manipulation: a robot must remember which object was placed in a drawer, whose cup it moved, or how many action cycles have elapsed. Recent vision-language-action (VLA) models embed memory directly *inside* the policy, but benchmarks show no single in-policy mechanism covers all spatio-temporal dimensions, trailing oracle methods by a wide margin. We argue that memory *type* dictates where memory should reside: short-term perceptual memory (repetition, timing, retracing) belongs *inside* the policy, while long-term object memory (persistent spatial state, containment, event history) belongs *outside* as an explicit, readable record. We present **LEDGER**, a harness that realizes this split over a single fine-tuned $\pi_{0.5}$ policy by pairing an in-policy frame-sampling memory with an external spatio-temporal object memory, the ledger (a SAM3 tracker, a VLM captioner of the demonstration, and an LLM that decides at step boundaries). On RoboMME, LEDGER reaches the highest four-suite average among the evaluated methods, 64.3% (vs. 45.9% for the strongest prior method under identical evaluation), leading object reference (60.7% vs. 40.3%) and object permanence (86.7% vs. 56.2%) using a single set of weights. Choosing the memory source at runtime, from the instruction and the record, removes the need for a task-level router.

1 Introduction

Memory allows humans to navigate and reason over partially-observed environments. For instance, if Alice wants to make a coffee latte using Bob’s cup, she would have to remember where the coffee grounds are, the milk’s expiry date, and where and which cup belongs to Bob. The information that decides the next action is no longer in the current frame. However, naively conditioning policies on long sequences of observations is computationally expensive, and policies start relying on spurious correlations in the history tokens instead of the immediate physical layout [11]. On RoboMME [5], a large-scale benchmark for memory-augmented robotic manipulation, the best in-policy memory reaches 44.5% averaged over four memory suites as reported (45.9% under our reproduction in Sec. 4), while the same backbone trained on the same demonstrations reaches 84.1% when an oracle hands it a correctly resolved target. These results indicate that resolving what the policy has to act on is a major remaining bottleneck, rather than low-level execution alone.

We start from the observation that the tasks in such benchmarks demand two different kinds of memory. *Short-term perceptual memory* is what the robot itself saw and did: how many swings it has completed, where the stick has already been, what the demonstrated motion looked like. This memory preserves rich low-level visual information that is difficult to capture with language, and is integrated inside the VLA. *Long-term object memory* is what the world did: which object sits where now, which container was placed over which cube, which cube was picked earlier, and in what order objects were placed. This memory is persistent, outside the policy as an explicit record that can be read, checked, and reasoned over, and must survive occlusion and distractors.

This raises a routing question: which memory should a given instruction use? Existing RoboMME methods do not ask

which the authors conclude that memory representations are complementary rather than exclusive. LIBERO-Mem [4] makes the object-centric case with visually identical bowls that only their interaction history tells apart; even its own slot-based policy, given oracle per-object subgoals, completes only 14.8% of subgoals, and π_0 completes 5%. RMBench [3] defines memory complexity as the number of past observations that an optimal policy must retain. Its Mem-0 baseline improves $\pi_{0.5}$ from 10.4% to 42.0% using a planner invoked at subtask boundaries over a memory of completed subtasks, with groundtruth signals worth 17 points more. RoboMemArena [16] tests horizons of over a thousand steps, where frozen VLM planners collapse to 8.7%. Together, these benchmarks fix *what* must be remembered and the gaps, but leave open *how* memory should be represented, *where* it should live, and *which* representation serves *which* kind of memory. We use RoboMME because it spans four memory types with one embodiment and one training set.

Memory inside the policy. The in-policy memory variants from RoboMME compress the observation history in different ways. Uniform frame sampling can modulate the action expert, token dropping removes temporal redundancy in image patches, and recurrent memory compresses the visual token history sequence into fixed-size latent states carried through recurrent updates [1]. HAMLET [15] compactly encodes perceptual information at each step into moment tokens, integrated as memory features used during action prediction, and Gated Memory Policy [9] employs a learned memory gate mechanism that decides when to cross-attend a fixed window. SAM2Act+ [7] stores the policy’s own past action heatmaps in a fixed queue and cannot retain semantic information such as color. EchoVLA [17] fuses a voxel scene memory and a token queue into mobile-manipulation diffusion heads and degrades under dynamic occlusions. $\pi_{0.6}$ -MEM [22] and $\pi_{0.7}$ [14] combine a short and long-term memory, using compressed video and text representations. MemER [20], the strongest prior method on permanence and reference, proposes a hierarchical policy framework where a high-level policy uses selected keyframes and the most recent frames when producing text instructions for a low-level policy to execute; it falls on RoboMME to 38% and 21% on the swap tasks where oracle grounding reaches 99% and 80%. These models remember what the robot saw, but none keeps, per object, where it rested and what came to cover it.

Explicit records and harnesses. Object maps and scene graphs [10, 12] provide an interface for inference and reasoning, but only keep current information: ConceptGraphs relocates a moved object by asking an LLM for plausible containers, not by consulting where it last rested. Compose by Focus [18] conditions diffusion-based atomic skills on task-relevant 3D scene graphs and composes them with a VLM planner, improving robustness to distractors and novel scene compositions. Systems that place a reasoning model over a fixed policy expose state the same way. THEA [23] keeps a persistent symbolic scene graph, re-renders it into the planner’s context at every turn, and admits execution-derived edits only once an evaluator has confirmed a tool’s post-condition, yet the graph is a present-tense snapshot with a freshness stamp: it records neither where an object rested earlier nor what came to cover it. Pigey [8] lifts a frozen $\pi_{0.5}$ from 12.8% to 53.3% on LIBERO-PRO with a perceive-act-verify loop whose memory is the history of attempted subgoals, their outcomes, and whose verification is a gripper sensor plus the planning VLM itself. Robot Critics [21] finds frontier VLMs near chance (54.7% against 50%) at fine-grained success judgment. Finally, which memory a task needs has so far been decided before execution. DIRECT [6] trains a router on the per-task success and cost of a fixed pool of RoboMME memories and uses it once per episode from the initial observation and instruction. LEDGER keeps the record these systems lack: what happened in the scene as stationary intervals and event descriptions per object, confirming step boundaries from proprioception rather than a VLM verdict, and letting the planner decide, from the instruction and the ledger, whether to ground execution.

3 LEDGER

3.1 Preliminaries

Problem Setting. We consider language-conditioned manipulation policies trained to model the conditional distribution $\pi(\mathbf{A}_t | o_t, l_t)$, where $\mathbf{A}_t = [a_t, a_{t+1}, \dots, a_{t+H-1}]$ is an action chunk modeled from the current timestep t up to H timesteps, o_t is the robot’s current sensor observation, and l_t is the language instruction. At each timestep t , the robot receives a multi-view RGB observation $\mathbf{I}_t = [I_t^1, I_t^2, \dots, I_t^n]$, and $\mathbf{q}_t$ are the proprioceptive inputs from the robot (joint angles and gripper state), hence $o_t = [\mathbf{I}_t, \mathbf{q}_t]$. The demonstration of a task, the covering of an object, or the number of repetitions completed so far are not recoverable from o_t alone.

In order to execute complex, long-horizon tasks, we follow the hierarchical factorization [19] of the robot policy into a low-level control policy π_{LL} and a high-level policy π_{HL} . The low-level policy only carries a short-term perceptual memory (Sec. 3.3), while the high-level policy carries the long-term object-centric memory m_t (Sec. 3.2) and decides what the low-level policy should act on by giving it language subgoals (Sec. 3.4). We decompose action prediction as

$$\pi(a_{t:t+H}, l_{t+1}, m_{t+1} | o_{t-T:t}, m_t, g) \approx \pi_{\text{LL}}(a_{t:t+H} | o_{t-K:t}, l_{t+1}, g) \pi_{\text{HL}}(l_{t+1}, m_{t+1} | o_t, m_t, g),$$

where g is the task instruction, l_{t+1} is the subgoal handed to the low-level policy (possibly empty), and $K \ll T$.

3.2 Long-Term Memory: Object-Centric Spatio-Temporal

Long-term memory enters only through the high-level policy’s language input, as a record we call the *ledger*. Let $\mathcal{O}$ be the set of objects enumerated in the initial observation, and for each $i \in \mathcal{O}$, let $p_i(t) \in \mathbb{R}^2 \cup \{\perp\}$ denote its tracked image coordinate at timestep t ($\perp$ while its track is lost) across both the optional task demonstration and the execution rollout. The ledger at time t is formulated as

$$\mathcal{L}_t = (\{(d_i, p_i(t), \mathcal{R}_i(t), c_i)\}_{i \in \mathcal{O}}, \mathcal{N}), \quad (1)$$

where d_i is the textual object description, c_i is the timestamp at which object i was covered by an entity other than the robot arm (or $\emptyset$), $\mathcal{N}$ is the action transcript of the demonstration (or $\emptyset$), and $\mathcal{R}_i(t)$ is the list of *stationary intervals* of object i : the maximal time windows over which its track stays still, each stored with its start, its end, and its mean position $\bar{p}_i$. Spatial reasoning then reduces to one geometric primitive, co-location: two positions are the same place when they lie closer than a tolerance of under one object width, and every spatial query the planner evaluates is derived from it. Object j was placed onto i if a stationary interval of j that follows a placement is co-located with $\bar{p}_i$, the number of such intervals is the number of placements, and object i is currently covered by k if c_i is set and $p_k(t)$ is co-located with $p_i(t)$.

Building the ledger. At the start of an episode, the planner model enumerates the visible objects in the initial frame. A SAM3 tracker [2] then tracks each instance across both the demonstration video (when available) and the live execution rollout. Textual descriptions, coordinates, visibility flags, and stationary intervals are computed directly from these trajectories. Cover timestamps c_i are maintained by the tracker’s recovery gate: whenever a trajectory is interrupted, the planner model is shown the annotated frame and asked whether the object is hidden by the robot arm or by another entity. Occlusions caused by the arm trigger a wait state, whereas environmental occlusions record the timestamp c_i . The recovered trajectory subsequently follows the covering entity, ensuring the coordinate of a covered object continuously tracks the location of its cover. This mechanism explicitly captures spatial state modifications, such as container swaps, that static, single-frame scene graphs cannot represent. Appendix D illustrates an instance of the ledger provided to the high-level planner.

The action transcript. Pure geometric tracking cannot determine action semantics: which object was grasped, placed, or actuated, and in what sequence. When an episode includes a demonstration video, a video VLM (Qwen3-VL-30B) analyzes frames annotated with tracked instance identifiers to produce an ordered natural-language account of the robot’s execution alongside press-timing intervals. This action transcript is generated once per episode. For tasks without a demonstration video, the transcript is marked empty and the planner relies entirely on real-time observations. Empirically, the transcript reliably captures event sequences, temporal ordering, and repetition counts, but is less accurate at disambiguating destinations among visually identical objects; Appendix D shows an instance. The high-level planner is therefore instructed to extract event structure from the action transcript while resolving all spatial target coordinates via stationary-interval co-location.

3.3 Short-Term Memory: Frame Sampling as Feature Modulation

The low-level control policy is based on $\pi_{0.5}$ [13], where a vision-language backbone encodes input image observations and language prompts into visual-textual tokens $\mathbf{u}_t$. A flow-matching action expert then denoises a noisy action chunk $\mathbf{s}_t^0$ across L residual layers (omitting explicit denoising timestep conditioning for brevity):

$$\tilde{\mathbf{s}}_t^k = \mathbf{s}_t^{k-1} + \text{Attn}^k(\mathbf{s}_t^{k-1}, \mathbf{u}_t), \quad \hat{\mathbf{s}}_t^k = \gamma_t^k \odot \text{Norm}(\tilde{\mathbf{s}}_t^k) + \beta_t^k, \quad \mathbf{s}_t^k = \tilde{\mathbf{s}}_t^k + \text{MLP}^k(\hat{\mathbf{s}}_t^k), \quad (2)$$

where $(\gamma_t^k, \beta_t^k) = (1, 0)$ in the base, unmodulated model.

Short-term perceptual memory is integrated directly into this action expert. Following Dai et al. [5], the policy retains $N = 32$ uniformly sampled past observations $\mathcal{S}_t$. The vision encoder ϕ extracts features from each historical frame, which are then pooled into a compact token set $\mathbf{M}_t = [\text{Pool}(\phi(o_\tau))]_{\tau \in \mathcal{S}_t}$. Every expert layer queries these visual tokens via cross-attention to compute feature modulation parameters:

$$(\gamma_t^k, \beta_t^k) = (1, 0) + f^k(\text{Attn}_{\text{mod}}^k(\tilde{\mathbf{s}}_t^k, \mathbf{M}_t)), \quad (3)$$

where f^k is initialized close to zero, so that fine-tuning starts from the unmodulated policy. This in-policy memory provides an explicit execution trace of recent robot observations and joint actions, such as swing counts, trajectory histories, and motion patterns. However, it cannot maintain persistent object identities across severe occlusions or keep an explicit record of where each object was moved.

Long-term memory, by contrast, is supplied to the same low-level policy exclusively through the language channel as a grounded subgoal prompt. To enable a single unified policy to operate across both memory modalities, we introduce a *task-conditional subgoal dropout curriculum* during fine-tuning. On temporal and procedural benchmark suites, subgoals are dropped during training with probability 0.9, forcing the policy to execute tasks relying solely on its internal perceptual memory. On spatial and object suites, subgoals are dropped with probability 0.15, preserving the policy’s capacity to leverage external spatial targets for occluded objects.

Both prompt conditions, with and without a grounded subgoal, are thus in-distribution at deployment, and the planner selects between them at runtime. The suite labels serve only in training; deployment receives no task or suite identifier.

Without this curriculum the subgoal is a training shortcut on procedural tasks: it prescribes the motion step by step (“move backward, move right”), so the policy reads the answer from text and never builds the perceptual memory it must use once the subgoal is withheld.

3.4 Deciding at Step Boundaries

The high-level policy is an LLM planner (Claude Sonnet 5) that operates over the ledger and one annotated frame per decision, never the video stream or the motor commands. It is invoked at the start of an episode and whenever an execution step completes. Each query supplies the planner with the task instruction, the current ledger, the action transcript (if available), an annotated observation frame marking tracked objects with identifier letters, and an execution log listing completed steps alongside their completion timestamps. The planner returns a structured plan consisting of a sequence of subgoals, where each subgoal uses a sentence template from the low-level policy’s training vocabulary with a slot for a spatial coordinate target. Only the immediate first subgoal is executed, and subsequent decisions are deferred until the low-level policy completes the step.

Replies are parsed strictly: a plan must draw its sentences from the low-level vocabulary, alternate picks and places so that the gripper never holds two objects, and reference only tracked identifiers. An invalid reply is re-prompted with the validation error, and an episode that keeps failing validation is recorded as a failure rather than executed on a guess.

The planner may also emit an *empty* subgoal, which hands the rest of the episode to the low-level policy’s own memory (Sec. 3.3); grounding or deferring is thus an explicit planner action, taken from the instruction and the ledger.

Why step boundaries. A static plan generated at episode onset cannot adapt to downstream events, such as an object becoming visible only after a button press or containers swapping during manipulation. Conversely, revising plans at fixed time intervals or on visual changes induces instabilities caused by scene changes during execution. Evaluating decisions strictly at step completion mitigates both issues: replanning occurs at discrete execution boundaries when the gripper reaches a known, stable state, avoiding mid-motion intervention.

Step completion from proprioception. Step completion is determined entirely from robot proprioception rather than visual trackers or VLM evaluators. The gripper aperture is categorized into three discrete regimes—shut, hold, and open—normalized relative to the maximum observed aperture. A grasp corresponds to entering the hold regime, a release to transitioning from hold to open, and a press to a reach and retreat of the arm with the gripper shut. Regime transitions must persist across consecutive control steps to prevent false triggers, with extended temporal filtering applied to the hold state to ignore transient aperture states during full gripper closure. Each planned step specifies its terminating event type, and live target coordinates are queried from the tracker at the exact onset of step execution.

Table 1 Main results. RoboMME success rate (%) on the test split, 50 episodes per task, mean over evaluation seeds; Avg is the mean over the 16 tasks. Upper block: as reported by Dai et al. [5]. Lower block: methods run by us under the protocol of Sec. 4. Oracle and QwenVL are GroundSG with ground-truth subgoals and with the benchmark’s fine-tuned Qwen3-VL subgoal generator; FrameSamp is FrameSamp+Modul; Tracker is the tracker-only variant of our harness. Red marks the best method within each block; Human and Oracle are reference rows. Seed spreads are in Appendix C.

Method	Counting				Permanence				Reference				Imitation				Avg
	Bin	Pick	Swing	Stop	Video	Button	Video	Button	Pick	Video	Video	Video	Move	Insert	Pattern	Route	
	Fill	Xtimes	Xtimes	Cube	Umsk	Umsk	UmskS	UmskS	HighL	Repick	PlcBtn	PlcOrd	Cube	Peg	Lock	Stick	
As reported by Dai et al. [5]: three seeds $\times$ three checkpoints																	
Human	96.0	100.0	80.0	78.0	90.0	92.0	92.0	90.0	92.0	92.0	98.0	90.0	90.0	98.0	84.0	86.0	90.5
Oracle	85.8	100.0	100.0	49.7	98.8	95.0	99.2	80.2	83.3	97.3	100.0	100.0	87.8	15.6	97.0	55.6	84.1
QwenVL	52.0	92.7	7.3	0.0	88.7	24.0	30.7	14.0	15.1	25.3	54.0	31.8	71.6	3.3	6.7	6.0	32.7
FrameSamp	39.6	87.3	92.0	42.0	32.7	25.1	24.4	18.2	22.9	30.4	60.0	32.0	77.8	7.6	53.6	66.7	44.5
MemER	56.7	79.3	59.3	0.0	81.3	72.0	38.0	21.3	70.7	25.3	30.0	26.0	82.7	6.7	16.7	12.0	42.4
Reproduced by us: released final checkpoint, three seeds, one protocol																	
FrameSamp	44.0	89.3	96.0	47.3	34.0	26.7	24.0	18.0	20.0	33.3	58.0	38.0	81.3	5.3	55.3	63.3	45.9
MemER	55.3	78.0	61.3	2.0	84.7	82.0	36.0	22.0	73.3	26.0	30.7	31.3	80.7	3.3	16.7	8.7	43.3
Tracker	41.3	90.0	92.7	40.0	92.7	88.0	92.0	76.0	18.7	28.0	53.3	30.7	76.0	7.3	41.3	64.0	58.3
LEDGER	50.0	92.7	88.0	36.7	92.7	90.0	91.3	72.7	55.3	60.7	62.7	64.0	60.7	10.7	42.0	58.7	64.3

4 Experimental Evaluation

Experimental protocol. All methods are evaluated on the official RoboMME test split across 16 tasks, 50 evaluation episodes per task, and three random seeds. Results are reported using the final model checkpoint for each method: our fine-tuned policy at 80k steps, alongside the released final checkpoints for FrameSamp+Modul and MemER. Success rates represent mean performance across all evaluation seeds.

In addition to prior baselines, Table 1 includes a *tracker-only* ablation variant of our harness. On spatial and object tasks (Permanence and Reference), a fine-tuned VLM selects among SAM3 tracks without access to historical event records; on temporal and procedural tasks (Counting and Imitation), the low-level $\pi_{0.5}$ policy executes directly using its internal short-term memory without subgoals. This baseline explicitly isolates the gains provided by persistent geometric tracking prior to integrating historical event memory.

4.1 One Policy, Best Across Memory Types

Table 1 reports success rates across all four RoboMME memory benchmark suites. Prior methods that demonstrate strength on a single suite exhibit severe performance degradation on others: under our evaluation protocol, FrameSamp+Modul achieves 69.2%, 25.7%, 37.3%, and 51.3% on Counting, Permanence, Reference, and Imitation, respectively, whereas MemER obtains 49.2%, 56.2%, 40.3%, and 27.3%.

In contrast, LEDGER uses a unified set of weights and a single high-level planner across all four suites. On Counting and Imitation, LEDGER trails the suite-specific leader, FrameSamp+Modul, by two points (66.8% vs. 69.2%) and by eight points (43.0% vs. 51.3%), with both methods running on the same in-policy perceptual memory.

On Permanence and Reference, external object memory provides the largest gains: Permanence reaches 86.7% (vs. 56.2% for MemER), with the first three tasks ranging from 90.0% to 92.7% and ButtonUnmaskSwap reaching 72.7%. Reference achieves 60.7% (vs. 40.3%), with individual task success rates spanning 55.3% to 64.0%. Across the three seeds, the standard deviation of LEDGER’s suite means is 2.8, 3.5, 5.2, and 2.2 points on Counting, Permanence, Reference, and Imitation, and 0.9 points on the 16-task average (Appendix C).

4.2 The Optimal Memory Source is Task-Dependent

Table 1 also says *where* memory should reside. Read by information source, its lower block compares four: FrameSamp acts from in-policy frame memory with no subgoal, Tracker grounds a subgoal from the current tracks with no record of the past, LEDGER grounds it from the ledger, and the Oracle row is the ceiling of a ground-truth subgoal. The pattern is consistent across suites:

Table 2 Ablation of the decision loop on the demonstration-guided tasks (success %, one evaluation seed, 50 episodes per cell, checkpoint 80k; a cell’s binomial standard error is up to 7 points). The last row is the configuration of Table 1, which averages three seeds. Re-deciding at step boundaries lets the planner react to events that occur during execution; carrying the remaining plan across decisions keeps the count of completed repetitions.

Decision loop over the ledger	Unmask	UnmaskSwap	Repick	PlaceButton	PlaceOrder
Plan once at the start of the episode	90	68	60	58	60
+ cover state, re-decide after each step	92	86	28	66	66
+ remaining plan carried across decisions	92	92	54	66	56

- **Spatial (Permanence):** a tracked target is enough. Tracker and LEDGER reach 87.2% and 86.7% against 25.7% without a subgoal and a 93.3% oracle, and both score at least 88% on the first three Permanence tasks. Tracking resolves a persistent target, so spatial memory belongs *outside* the policy.
- **Temporal (Counting) and Procedural (Imitation):** the policy’s own memory is the best source. FrameSamp, which never receives a subgoal, leads both suites (69.2% and 51.3%), while the benchmark’s subgoal-driven pipeline trails far behind (38.0% and 21.9% for QwenVL). A grounded target is not what these tasks lack: on Counting it neither helps nor hurts (66.8% for LEDGER against 66.0% for the same policy without subgoals), and on MoveCube, where our planner grounds a target instead of deferring, it costs fifteen points (60.7% vs. 76.0%). Repetitions, timing, and motion patterns are not a coordinate, so temporal and procedural memory belong *inside* the policy.
- **Object (Reference):** neither source suffices alone. Tracks with no record of the past give 32.7% and in-policy memory 37.3%, against a 95.2% oracle; the ledger lifts the suite to 60.7%. The policy can execute the skill given the right target, but identifying that target requires a record of past events, which is what Sec. 3.2 supplies.

Given a correctly resolved target, the benchmark’s oracle-conditioned policy executes well on every suite (Oracle: 83.9%, 93.3%, 95.2%, and 64.0%), so what remains is *resolving* what to act on and when, which LEDGER does according to memory type.

What the decision loop contributes. Table 2 varies the decision loop of Sec. 3.4 over the ledger of Sec. 3.2 on the five demonstration tasks. With a single plan made at the start of the episode, the planner resolves persistent targets (Unmask 90%) but cannot react to what happens during execution. Re-deciding from scratch after every completed step, with the cover state exposed, lifts the two tasks whose key event occurs after execution starts, PlaceButton and PlaceOrder (58% → 66% and 60% → 66%), but destabilizes Repick (60% → 28%): without a record of its own earlier decisions, the planner re-attributes completed repetitions to the remaining goal and miscounts. Carrying the remaining plan across decisions and counting repetitions from the execution log restores Repick to 54% and keeps PlaceButton at 66%. On the container-swap task UnmaskSwap, re-deciding lifts success from 68% to 86% and carrying the plan to 92%.

Can the high-level policy choose the memory source at runtime? Whether to ground a target or to defer to the policy’s own memory is a planner action, taken from the instruction and the ledger and never from task or suite metadata. We evaluate it in the complete closed loop, with the planner deployed on all 16 tasks. Table 1 reports this always-on configuration, and on Counting and Imitation its Tracker row is the same policy in memory mode, so the comparison reads directly off the table. The planner defers on every *PatternLock* and *RouteStick* episode and on two thirds of *StopCube* episodes, where deferring succeeds more often than grounding, and stays within six points of memory mode on all three (42.0% vs. 41.3%, 58.7% vs. 64.0%, and 36.7% vs. 40.0%). Where the instruction names an object to act on, it grounds instead: on *BinFill* and *PickXtimes* it plans the picks, the places, and the final press, counting them from confirmed step completions, and improves on memory mode (50.0% vs. 41.3% and 92.7% vs. 90.0%), so that Counting as a whole matches memory mode (66.8% vs. 66.0%). The deficit on Imitation (43.0% vs. 47.2%) stems mostly from *MoveCube*, where the planner grounds a coordinate target for a motion that should be reproduced from memory (60.7% vs. 76.0%). On the benchmark’s instructions, the planner thus infers the memory allocation at runtime from the instruction and the ledger, at the cost of one task.

5 Discussion and Future Work

While LEDGER leads the evaluated methods across diverse memory modalities, several limitations remain. First, our implementation operates in simulation, grounding targets in 2D image coordinates. Extending this architecture to mobile manipulation will require 3D spatial grounding and camera pose tracking. Second, action transcripts can introduce errors: on tasks requiring ordinal placement matching (e.g., VideoPlaceOrder), the ledger reaches 64.0% against 100% for oracle grounding, as temporal event semantics cannot be verified via geometry alone.

Despite these limitations, externalizing persistent world state into an explicit ledger provides clear advantages. Transparent object records make high-level reasoning interpretable and allow direct human intervention, such as updating object properties without modifying policy weights. Future work will focus on 3D mobile manipulation, real-world deployment, dynamic mid-episode object discovery, and further reducing supervisory fine-tuning as foundational VLMs evolve.

6 Conclusion

Where does each kind of memory belong? Short-term perceptual memory belongs inside the policy, as history-conditioned features, long-term spatial and event memory belongs outside the policy as an explicit spatio-temporal record, and a high-level planner chooses between them at runtime. LEDGER realizes this split by pairing $\pi_{0.5}$ with a tracker-maintained ledger, a VLM-generated action transcript, and an LLM planner deciding at proprioception-confirmed step boundaries. On RoboMME, it is the only evaluated method competitive on all four suites and reaches the highest average success among them, 64.3%.

Acknowledgments

This work was partially funded by Office of Naval Research grant N00014-25-1-2322.

References

- [1] Aydar Bulatov, Yuri Kuratov, and Mikhail Burtsev. Recurrent memory transformer. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=Uynr3iPhksa>. 3
- [2] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris Coll-Vinent, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, Jie Lei, Tengyu Ma, Baishan Guo, Arpit Kalla, Markus Marks, Joseph Greer, Meng Wang, Peize Sun, Roman Rädle, Triantafyllos Afouras, Effrosyni Mavroudi, Katherine Xu, Tsung-Han Wu, Yu Zhou, Liliane Momeni, RISHI HAZRA, Shuangrui Ding, Sagar Vaze, Francois Porcher, Feng Li, Siyuan Li, Aishwarya Kamath, Ho Kei Cheng, Piotr Dollar, Nikhila Ravi, Kate Saenko, Pengchuan Zhang, and Christoph Feichtenhofer. SAM 3: Segment anything with concepts. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=r35clVtGzw>. 4
- [3] Tianxing Chen, Yuran Wang, Mingleyang Li, Yan Qin, Hao Shi, Zixuan Li, Yifan Hu, Yingsheng Zhang, Kaixuan Wang, Yue Chen, et al. Rmbench: Memory-dependent robotic manipulation benchmark with insights into policy design. *arXiv preprint arXiv:2603.01229*, 2026. 3
- [4] Nhat Chung, Taisei Hanyu, Toan Nguyen, Huy Le, Frederick Bumgarner, Duy Minh Ho Nguyen, Khoa Vo, Kashu Yamazaki, Chase Rainwater, Tung Kieu, Anh Nguyen, and Ngan Le. Rethinking progression of memory state in robotic manipulation: An object-centric perspective. *arXiv preprint arXiv:2511.11478*, 2025. 3
- [5] Yinpei Dai, Hongze Fu, Jayjun Lee, Yuejiang Liu, Haoran Zhang, Jianing Yang, Chelsea Finn, Nima Fazeli, and Joyce Chai. Robomme: Benchmarking and understanding memory for robotic generalist policies. *arXiv preprint arXiv:2603.04639*, 2026. 1, 2, 5, 6
- [6] Jadelynn Dao, Milan Ganai, Yasmina Abukhadra, Ajay Sridhar, Mozghan Nasr Azadani, Katie Luo, Clark Barrett, Jiajun Wu, Chelsea Finn, and Marco Pavone. Direct: When and where should you allocate test-time compute in embodied planners?, 2026. URL <https://arxiv.org/abs/2606.12402>. 2, 3

- [7] Haoquan Fang, Markus Grotz, Wilbert Pumacay, Yi Ru Wang, Dieter Fox, Ranjay Krishna, and Jiafei Duan. SAM2act: Integrating visual foundation model with a memory architecture for robotic manipulation. In *7th Robot Learning Workshop: Towards Robots with Human-Level Abilities*, 2025. URL <https://openreview.net/forum?id=XuUNUXVgmb>. 3
- [8] Liane Galanti, Dhruv Shah, and Tri Dao. Addressing the orchestration gap in generalist robots via physical agency. *arXiv preprint arXiv:2607.21725*, 2026. 3
- [9] Yihuai Gao, Jinyun Liu, Shuang Li, and Shuran Song. Gated memory policy, 2026. URL <https://arxiv.org/abs/2604.18933>. 3
- [10] Qiao Gu, Ali Kuwajerwala, Sacha Morin, Krishna Murthy Jatavallabhula, Bipasha Sen, Aditya Agarwal, Corban Rivera, William Paul, Kirsty Ellis, Rama Chellappa, et al. Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5021–5028. IEEE, 2024. 3
- [11] Jisang Han, Seonghu Jeon, Jaewoo Jung, René Zurbrügg, Honggyu An, Tifanny Portela, Marco Hutter, Marc Pollefeys, Seungryong Kim, and Sunghwan Hong. Geometric action model for robot policy learning, 2026. URL <https://arxiv.org/abs/2606.17046>. 1
- [12] N. Hughes, Y. Chang, and L. Carlone. Hydra: A real-time spatial perception system for 3D scene graph construction and optimization. 2022. 3
- [13] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025. URL <https://arxiv.org/abs/2504.16054>. 2, 4
- [14] Physical Intelligence, Bo Ai, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Greg Balke, Kevin Black, George Bokinsky, Shihao Cao, Thomas Charbonnier, Vedant Choudhary, Foster Collins, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Maitrayee Dhaka, Jared DiCarlo, Danny Driess, Michael Equi, Adnan Esmail, Yunhao Fang, Chelsea Finn, Catherine Glossop, Thomas Godden, Ivan Goryachev, Lachlan Groom, Haroun Habeeb, Hunter Hancock, Karol Hausman, Gashon Hussein, Victor Hwang, Brian Ichter, Connor Jacobsen, Szymon Jakubczak, Rowan Jen, Tim Jones, Gregg Kammerer, Ben Katz, Liyiming Ke, Mairbek Khadikov, Chandra Kuchi, Marinda Lamb, Devin LeBlanc, Brendon LeCount, Sergey Levine, Xinyu Li, Adrian Li-Bell, Vladislav Lialin, Zhonglin Liang, Wallace Lim, Yao Lu, Enyu Luo, Vishnu Mano, Nandan Marwaha, Aikys Mongush, Liam Murphy, Suraj Nair, Tyler Patterson, Karl Pertsch, Allen Z. Ren, Gavin Schelske, Charvi Sharma, Baifeng Shi, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, Will Stoeckle, Jiaming Tang, Jimmy Tanner, Shalom Tekeste, Marcel Torne, Kyle Vedder, Quan Vuong, Anna Walling, Haohuan Wang, Jason Wang, XuDong Wang, Chris Whalen, Samuel Whitmore, Blake Williams, Charles Xu, Sukwon Yoo, Lili Yu, Wuming Zhang, Zhuoyang Zhang, and Ury Zhilinsky. $\pi_{0.7}$: a steerable generalist robotic foundation model with emergent capabilities, 2026. URL <https://arxiv.org/abs/2604.15483>. 3
- [15] Myungkyu Koo, Daewon Choi, Taeyoung Kim, Kyungmin Lee, Changyeon Kim, Younggyo Seo, and Jinwoo Shin. HAMLET: Switch your vision-language-action model into a history-aware policy. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=KcJ9U0x6k0>. 3
- [16] Huashuo Lei, Wenxuan Song, Huarui Zhang, Jieyuan Pei, Jiayi Chen, Haodong Yan, Han Zhao, Pengxiang Ding, Zhipeng Zhang, Lida Huang, Donglin Wang, Yan Wang, and Haoang Li. Robomemarena: A comprehensive and challenging robotic memory benchmark. *arXiv preprint arXiv:2605.10921*, 2026. 3
- [17] Min Lin, Xiwen Liang, Bingqian Lin, Liu Jingzhi, Zijian Jiao, Kehan Li, Yu Sun, Weijia Liufu, Yuhua Ma, Yuecheng Liu, Shen Zhao, Yuzheng Zhuang, and Xiaodan Liang. Echovla: Synergistic declarative memory for vla-driven mobile manipulation, 2026. URL <https://arxiv.org/abs/2511.18112>. 3
- [18] Han Qi, Changhe Chen, and Heng Yang. Compose by Focus: Scene graph-based atomic skills. In *2026 IEEE International Conference on Robotics and Automation (ICRA)*, 2026. URL <https://arxiv.org/abs/2509.16053>. 3
- [19] Lucy Xiaoyang Shi, brian ichter, Michael Robert Equi, Liyiming Ke, Karl Pertsch, Quan Vuong, James Tanner, Anna Walling, Haohuan Wang, Niccolo Fusai, Adrian Li-Bell, Danny Driess, Lachy Groom, Sergey Levine, and Chelsea Finn. Hi robot: Open-ended instruction following with hierarchical vision-language-action models. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=lNVHg9npif>. 4
- [20] Ajay Sridhar, Jennifer Pan, Satvik Sharma, and Chelsea Finn. Memer: Scaling up memory for robot control via experience retrieval. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2510.20328>. 3

- [21] Sruthi Sudhakar, Junbang Liang, Sreehari Rammohan, Pavel Tokmakov, Richard Zemel, and Carl Vondrick. Robot critics that sweat the small stuff, 2026. URL <https://arxiv.org/abs/2606.21572>. 3
- [22] Marcel Torne, Karl Pertsch, Homer Walke, Kyle Vedder, Suraj Nair, Brian Ichter, Allen Z. Ren, Haohuan Wang, Jiaming Tang, Kyle Stachowicz, Karan Dhabalia, Michael Equi, Quan Vuong, Jost Tobias Springenberg, Sergey Levine, Chelsea Finn, and Danny Driess. Mem: Multi-scale embodied memory for vision language action models, 2026. URL <https://arxiv.org/abs/2603.03596>. 3
- [23] Qi Wang, Tianyi Wang, Chengyang Li, Shikun Ban, Yurun Chen, Yizhong Ge, Jason Qin, Chengtai Li, and Wentao Zhu. Towards the harness of embodied agents, 2026. Technical Report. 3

A The sixteen RoboMME tasks

Table 3 lists one instruction per task; Figure 2 shows a first frame of each.

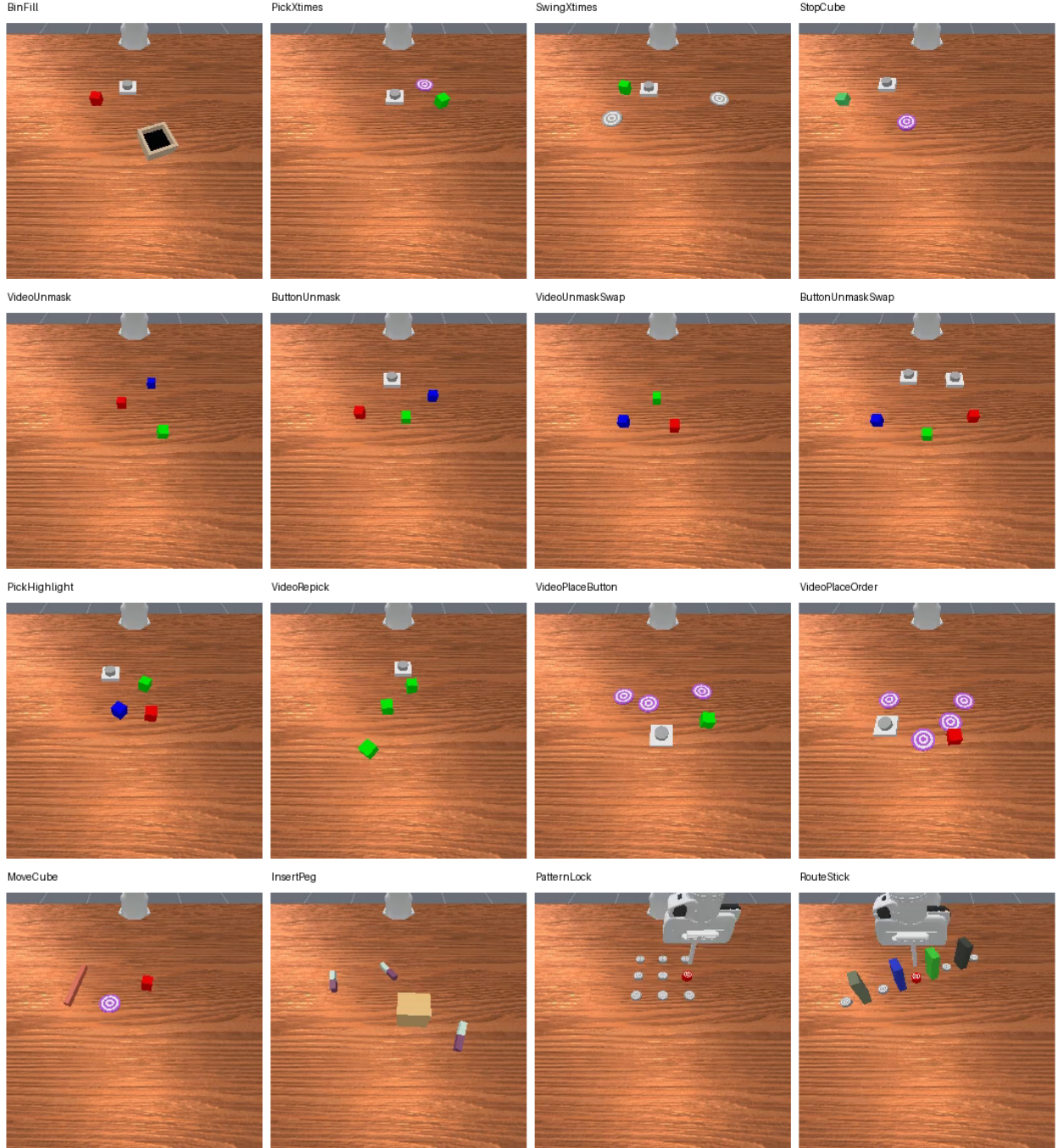


Figure 2 First frame of one test episode of each RoboMME task, from our evaluation runs (front camera, 256×256 , the resolution the policy and the tracker see). Rows are the four suites: Counting, Permanence, Reference, Imitation. Table 3 gives the instruction of each task.

Table 3 One instruction instance per task, as given to the planner and to the low-level policy; object colours, counts, and targets vary across episodes. Tasks that begin with “watch the video carefully” start with a demonstration video; the others start from a single observed frame.

Task	Instruction
BinFill	put one red cube into the bin, then press the button to stop
PickXtimes	pick up the green cube and place it on the target, repeating this action three times, then press the button to stop
SwingXtimes	pick up the green cube, move it to the top of the right-side target, then move it to the top of the left-side target, repeating this back-and-forth motion two times, finally press the button to stop
StopCube	press the button to stop the cube just as it reaches the target for the fourth time
VideoUnmask	watch the video carefully, then pick up the container hiding the green cube
ButtonUnmask	first press the button, then pick up the container hiding the red cube
VideoUnmaskSwap	watch the video carefully, then pick up the container hiding the green cube, finally pick up another container hiding the blue cube
ButtonUnmaskSwap	first press both buttons on the table, then pick up the container hiding the blue cube, finally pick up another container hiding the green cube
PickHighlight	first press the button, then pick up all cubes that have been highlighted [sic] with white areas on the table
VideoRepick	watch the video carefully, then repeatedly pick up and put down the same block that was previously picked up for three times, finally put it down and press the button to stop
VideoPlaceButton	watch the video carefully, then place the green cube on the target right after the button was pressed
VideoPlaceOrder	watch the video carefully, then place the red cube on the first target it was previously placed on
MoveCube	watch the video carefully, then move the cube to the target in the same manner as before
InsertPeg	watch the video carefully, then grasp the same end of the same peg you’ve picked before and insert it into the same side of the box
PatternLock	watch the video carefully, then use the stick attached to the robot to retrace the same pattern
RouteStick	watch the video carefully, then use the stick attached to the robot to navigate around the sticks on the table, following the same path

B Training and inference details

The low-level policy is a $\pi_{0.5}$ VLA (PaliGemma-2B backbone with a flow-matching action expert) fine-tuned in full from the public `p105_base` checkpoint with the standard flow-matching objective for 80k steps at batch size 64, under the subgoal dropout schedule of Sec. 3.3. The tracker is SAM3. The VLM captioner is Qwen3-VL-30B and takes about one minute per demonstration on one A100. Seeding, planning, and occlusion queries use Claude Sonnet 5.

Planner calls. The planner is queried 2.7 times per episode on average, from once on StopCube, PatternLock, and RouteStick to 4.8 times on PickXtimes.

C Seed variance

Table 4 gives the mean and standard deviation over the three evaluation seeds of each suite mean and of the 16-task average, for the rows of Table 1 that we ran ourselves.

Table 4 Mean $\pm$ standard deviation over the three evaluation seeds of the suite means and of the 16-task average (success %).

Method	Counting	Permanence	Reference	Imitation	Avg
FrameSamp	69.2 $\pm$ 2.9	25.7 $\pm$ 1.3	37.3 $\pm$ 3.2	51.3 $\pm$ 2.5	45.9 $\pm$ 0.7
MemER	49.2 $\pm$ 1.6	56.2 $\pm$ 0.6	40.3 $\pm$ 1.9	27.3 $\pm$ 2.0	43.3 $\pm$ 0.7
Tracker	66.0 $\pm$ 1.8	87.2 $\pm$ 5.1	32.7 $\pm$ 0.8	47.2 $\pm$ 0.8	58.3 $\pm$ 1.8
LEDGER	66.8 $\pm$ 2.8	86.7 $\pm$ 3.5	60.7 $\pm$ 5.2	43.0 $\pm$ 2.2	64.3 $\pm$ 0.9

D The ledger as the planner reads it

Below is the complete message the planner received at its first decision in one VideoUnmaskSwap episode with a single-container instruction, followed by its reply, taken from the run logs. Three cubes were seeded from the first

frame, the tracker followed them through the 169-frame demonstration, and all three were reported covered at tick 35, when the containers came down. The transcript is wrong: it describes the red cube being stacked on the green one. The stationary intervals allow for the recovery: between ticks 88 and 112 the covers over A and B exchanged places (A's second rest at (117.6, 86.3) is within near_px of B's first rest at (119.0, 93.4), and B's second rest is at A's first), and between ticks 142 and 162 they exchanged back. The planner needs none of this reasoning to act, as a covered object's position is where its cover sits now, so it grounds the pick at B's current position.

```
MEMORY:
{"objects": [
  {"id": "A", "description": "red cube (small red cube)", "position": [142.5, 73.3],
    "visible": true, "covered_since": 35,
    "rests": [{"t": [1, 99], "position": [142.5, 76.5]},
      {"t": [112, 141], "position": [117.6, 86.3]},
      {"t": [162, 170], "position": [142.5, 73.4]}]},
  {"id": "B", "description": "green cube (small green cube)", "position": [119.1, 93.4],
    "visible": true, "covered_since": 35,
    "rests": [{"t": [1, 88], "position": [119.0, 93.4]},
      {"t": [111, 142], "position": [144.8, 79.9]},
      {"t": [162, 170], "position": [119.1, 93.3]}]},
  {"id": "C", "description": "blue cube (small blue cube)", "position": [120.4, 127.8],
    "visible": true, "covered_since": 35,
    "rests": [{"t": [1, 170], "position": [120.4, 128.0]}]},
  "near_px": 12.8,
  "demonstration": "The robot arm lifts the red cube (A) and places it on top of the
    green cube (B), then lifts both cubes together and sets them down next to the blue
    cube (C). Finally, the robot arm presses down on the stacked cubes (A and B) while
    they remain in place beside C.\n\nPress timing, answered separately: There is no
    pressing action observed in the video. All movements involve lifting and placing
    the cubes, but none of the objects are pressed down by the robot arm. Therefore, no
    object is pressed, and there is no sequence of placements immediately before or
    after a press."}]
```

```
COMPLETED THIS EPISODE:
  none yet
```

```
YOUR REMAINING PLAN FROM THE PREVIOUS DECISION:
  none: this is the first decision
```

```
GRIPPER NOW: empty
```

```
Instruction: watch the video carefully, then pick up the container hiding the green cube
```

```
Resolve the referents from the memory and output the remaining steps.
```

The planner's reply, parsed strictly (a sentence from the executor vocabulary, one referent, a coordinate slot):

```
referents: B (green cube's container, identified by description/position; covered_since
shows a container sits over it)
```

```
{"steps": [{"action": "pick", "obj": "B",
  "subgoal": "pick up the container at <> that hides the green cube"}]}
```

The harness fills the slot with B's live coordinate and hands the sentence to the low-level policy; proprioception confirms the grasp at tick 257, and the second decision receives the same memory with the completed-steps block filled in and a gripper line stating that an object is held:

```
subgoal handed to the policy: pick up the container at <93, 119> that hides the green cube
COMPLETED THIS EPISODE (decision 2):
  1. pick up the container at <> that hides the green cube (obj B), confirmed at t=257
```

The planner replies with an empty step list, and the episode ends as a success. The message also carries an annotated current frame, in which each tracked object is marked with its letter, that is not reproduced here.